React Hooks

Advanced Guide — React 18 & 19

The 8 hooks missing from your interview prep — useTransition, useDeferredValue, useId, useSyncExternalStore, useActionState, useFormStatus, useOptimistic, and use.

1. **useTransition** — Mark state updates as non-urgent (React 18)
2. **useDeferredValue** — Defer a value to keep the UI responsive (React 18)
3. **useId** — Generate unique IDs stable across SSR (React 18)
4. **useSyncExternalStore** — Subscribe to external stores safely (React 18)
5. **useActionState** — Manage form state from server actions (React 19)
6. **useFormStatus** — Read parent form's pending status (React 19)
7. **useOptimistic** — Optimistic UI during async actions (React 19)
8. **use** — Read Promises and Context during render (React 19)

Prepared for Akshay — Interview Prep Series

#1

useTransition

Mark a state update as non-urgent — keep the UI responsive during expensive renders.

What is useTransition?

In plain English: useTransition tells React *'this state update is not urgent — if something more important comes along (like the user typing), handle that first.'* It splits your updates into two lanes: urgent (input changes, clicks) and non-urgent (filtering a list, switching tabs). The urgent update happens immediately; the non-urgent one can be interrupted and restarted.

Real-World Analogy

Think of a restaurant kitchen. An urgent order (appetizer for a VIP) gets prioritized. A non-urgent order (dessert for table 5) can wait. The kitchen doesn't stop entirely — it just reorders the queue. useTransition is the kitchen manager deciding what to cook first.

Syntax

```
const [isPending, startTransition] = useTransition();

// isPending      → boolean, true while transition runs
// startTransition → function to wrap non-urgent updates
```

Beginner — Keeping Input Responsive

The classic use case: a search input that filters a huge list. Without useTransition, every keystroke triggers an expensive filter, making the input feel laggy.

```
function SearchPage({ items }) {
  const [query, setQuery] = useState('');
  const [filtered, setFiltered] = useState(items);
  const [isPending, startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);              // URGENT: update input now

    startTransition(() => {
      setFiltered(               // NON-URGENT: can wait
        items.filter(i =>
          i.name.toLowerCase().includes(value.toLowerCase())
        )
      );
    });
  };

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <Spinner />}
      <List items={filtered} />
    </>
  );
}
```

Intermediate — Tab Switching

Switching between tabs that render expensive content. Without useTransition, clicking a tab freezes the UI until the heavy tab finishes rendering.

```
function TabContainer() {
  const [tab, setTab] = useState('home');
  const [isPending, startTransition] = useTransition();

  const selectTab = (nextTab) => {
    startTransition(() => {
      setTab(nextTab);    // expensive tab render is deferred
    });
  };

  return (
    <>
      <TabButton onClick={() => selectTab('home')}>Home</TabButton>
      <TabButton onClick={() => selectTab('analytics')}>
        Analytics
      </TabButton>
      <div style={{ opacity: isPending ? 0.6 : 1 }}>
        {tab === 'home' ? <Home /> : <Analytics />}
      </div>
    </>
  );
}
```

■■ **Key Insight:** useTransition does NOT make code faster — it makes the UI feel faster by keeping urgent updates responsive. The expensive work still happens; it just doesn't block user interaction.

Interview Q&A

Q: What's the difference between useTransition and setTimeout?

A: setTimeout defers work to a later time (macro-task). useTransition tells React's scheduler to treat the update as interruptible — React can pause it mid-render to handle urgent updates, then resume. setTimeout can't be interrupted once started. useTransition is React-aware; setTimeout is not.

Q: Can you use useTransition for data fetching?

A: Yes! In React 19 + Next.js App Router, wrapping a server action in startTransition is the pattern for non-blocking navigation and data mutations. Next.js router.push() internally uses transitions.

Q: Does isPending cause a re-render?

A: Yes. When the transition starts, isPending becomes true (re-render). When it finishes, isPending becomes false (another re-render). So you get 2 extra renders — use it only when the UX benefit justifies it.

#2

useDeferredValue

Defer a value so the UI stays responsive — the value version of useTransition.

What is useDeferredValue?

In plain English: useDeferredValue says *'give me a copy of this value that's allowed to lag behind.'* React will first render with the old (stale) value, then re-render in the background with the new value. If another urgent update arrives, React abandons the background render.

Real-World Analogy

Imagine a live cricket scoreboard. The main score (urgent) updates instantly. But the statistics panel (deferred) updates a few seconds later because it's less critical. You always see the score first.

Syntax

```
const deferredValue = useDeferredValue(value);

// value         -> the original (urgent) value
// deferredValue -> a copy that can lag behind
```

Example — Deferred Search Results

```
function Search({ query }) {
  const deferredQuery = useDeferredValue(query);
  const isStale = query !== deferredQuery;

  // This expensive computation uses the DEFERRED value
  const results = useMemo(
    () => heavySearch(deferredQuery),
    [deferredQuery]
  );

  return (
    <div style={{ opacity: isStale ? 0.5 : 1 }}>
      {results.map(r => <Item key={r.id} {...r} />)}
    </div>
  );
}
```

useTransition vs useDeferredValue

Aspect	useTransition	useDeferredValue
What you wrap	The state SETTER	The VALUE itself
Control	You decide WHEN to defer	React decides based on value changes
Returns	[isPending, startTransition]	deferredValue (lagging copy)
Best when	You own the setState call	You receive a prop you can't control
Loading indicator	isPending boolean built-in	Compare value !== deferredValue
Use case	Wrapping onClick, onChange handlers	Child component receives a prop

Interview Q&A

Q: When should you pick useDeferredValue over useTransition?

A: Use useDeferredValue when you can't wrap the state update — e.g., a prop passed from a parent, a value from a library hook, or a context value. useTransition requires you to wrap the setter, so you need ownership of the update. useDeferredValue works on any value.

Q: Does useDeferredValue cause extra renders?

A: Yes — React renders twice: once with the stale value (fast, reuses cached results), then once with the new value (background). The first render keeps the UI responsive. The second updates the content. React can abort the second render if a newer value arrives.

#3

useId

Generate unique IDs that are stable across server and client — no hydration mismatches.

What is useId?

In plain English: useId generates a unique string ID that's the same on the server (SSR) and the client (hydration). This solves a common Next.js problem: if you generate IDs with Math.random() or a counter, the server and client produce different values, causing hydration errors.

Real-World Analogy

Think of seat numbers on an airplane ticket. Your boarding pass (server) says seat 12A. When you board (client), the same seat 12A is waiting. useId ensures the server-generated ID and the client-generated ID always match — like pre-assigned seats.

Syntax

```
const id = useId();
// Returns something like ":r1:" or ":r4d:"
// Same value on server and client
```

Example — Form Labels

```
function FormField({ label }) {
  const id = useId();

  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} />
    </>
  );
}

// Multiple IDs from one useId call:
function PasswordField() {
  const id = useId();

  return (
    <>
      <label htmlFor={id + "-password"}>Password</label>
      <input id={id + "-password"} type="password" />
      <label htmlFor={id + "-confirm"}>Confirm</label>
      <input id={id + "-confirm"} type="password" />
    </>
  );
}
```

■■ **Common Mistake:** Do NOT use useId for list keys. useId generates one ID per component instance, not per list item. For list keys, use data IDs (database ID, slug, etc.) or the item index as a last resort.

Interview Q&A

Q: Why not just use Math.random() or a global counter for IDs?

A: Math.random() produces different values on server and client, causing hydration mismatches in Next.js. A global counter can produce different values if components render in different orders. useId is deterministic based on the component tree position — same on server and client, every time.

Q: Can you use useId for CSS selectors or querySelector?

A: Technically yes, but the colon in the ID (e.g., ':r1:') needs escaping in CSS: #\:r1\:. It works but it's clunky. useId is designed for HTML attributes (htmlFor, id, aria-describedby), not for CSS targeting.

#4

useSyncExternalStore

Subscribe to external (non-React) stores safely — works correctly with concurrent rendering.

What is useSyncExternalStore?

In plain English: useSyncExternalStore is for subscribing to state that lives OUTSIDE React — browser APIs (navigator.onLine, window.innerWidth), third-party stores, or global variables. It guarantees the UI never shows inconsistent ('torn') data during concurrent renders.

Real-World Analogy

Think of a stock ticker. The stock price (external store) changes independently of your app. useSyncExternalStore is like a secure feed — it subscribes to price changes and guarantees your app always shows one consistent price, never a mix of old and new.

Syntax

```
const value = useSyncExternalStore(
  subscribe,         // (callback) => unsubscribe fn
  getSnapshot,       // () => current value (client)
  getServerSnapshot // () => value for SSR (optional)
);
```

Example — Online Status

```
function useOnlineStatus() {
  return useSyncExternalStore(
    (callback) => {
      window.addEventListener("online", callback);
      window.addEventListener("offline", callback);
      return () => {
        window.removeEventListener("online", callback);
        window.removeEventListener("offline", callback);
      };
    },
    () => navigator.onLine,    // client snapshot
    () => true                 // server snapshot (assume online)
  );
}

// Usage
function StatusBar() {
  const isOnline = useOnlineStatus();
  return <p>{isOnline ? 'Online' : 'Offline'}</p>;
}
```

Example — Window Width

```
function useWindowWidth() {
  return useSyncExternalStore(
    (cb) => {
      window.addEventListener("resize", cb);
      return () => window.removeEventListener("resize", cb);
    },
    () => window.innerWidth,
    () => 1024  // fallback for SSR
  );
}
```

■■ **Practical Note:** You probably won't call useSyncExternalStore directly. Libraries like Zustand, Redux Toolkit, and Jotai use it internally. Know it for interviews and for understanding how state libraries work.

Interview Q&A

Q: Why not just use useEffect + useState for external subscriptions?

A: useEffect + useState can cause 'tearing' in concurrent mode — two parts of the UI reading different values from the same store during a single render. useSyncExternalStore guarantees consistency by forcing a synchronous read. It also handles SSR via the third argument.

Q: What is 'tearing' in React concurrent rendering?

A: Tearing happens when a render is interrupted, an external value changes, and the render resumes — resulting in some components reading the old value and others reading the new value. useSyncExternalStore prevents this by synchronizing reads.

#5

useActionState

Manage form state from server actions — the React 19 way to handle forms.

What is useActionState?

In plain English: useActionState connects a form to a server action (or any async function). It gives you the action's return value as state, a form-action function to pass to <form action={...}>, and an isPending flag. Previously called useFormState (renamed in React 19 RC).

Real-World Analogy

Think of submitting a bank deposit slip. You fill out the form (formData), hand it to the teller (server action), and wait for a receipt (state). The teller tells you if it succeeded or failed (return value). isPending is the 'processing' light on the counter.

Syntax

```
const [state, formAction, isPending] = useActionState(
  actionFn,     // (prevState, formData) => newState
  initialState  // initial state before first action
);
```

Complete Example — Next.js Server Action

```
// actions.ts (server)
"use server";

async function signup(prevState, formData) {
  const email = formData.get("email");
  const password = formData.get("password");

  if (!email.includes("@")) {
    return { error: "Invalid email", success: false };
  }

  await db.user.create({ data: { email, password } });
  return { error: null, success: true };
}
```

```
// SignupForm.tsx (client component)
"use client";
import { useActionState } from 'react';
import { signup } from './actions';

function SignupForm() {
  const [state, formAction, isPending] = useActionState(
    signup,
    { error: null, success: false }
  );

  return (
    <form action={formAction}>
      <input name="email" placeholder="Email" />
      <input name="password" type="password" />
      <button disabled={isPending}>
        {isPending ? 'Signing up...' : 'Sign Up'}
      </button>
      {state.error && <p className='error'>{state.error}</p>}
      {state.success && <p>Account created!</p>}
    </form>
  );
}
```

■■ **Why It Matters:** useActionState replaces the old pattern of useState + onSubmit + fetch + try/catch + loading state. One hook does all of it. This is the standard form pattern in Next.js App Router.

Interview Q&A

Q: What's the difference between useActionState and useFormStatus?

A: useActionState manages the form's state and action binding — it owns the data. useFormStatus only reads the pending status from a parent form — it owns nothing. useActionState is called in the component that renders the form. useFormStatus is called in a CHILD of the form.

Q: Why was useFormState renamed to useActionState?

A: Because it works with ANY action (not just forms). You can use it with server actions, client-side async functions, or even synchronous reducers. The 'Form' in the old name was misleading — 'Action' better describes what it does.

#6

useFormStatus

Read the pending status of a parent form — from inside a child component.

What is useFormStatus?

In plain English: useFormStatus lets a component inside a <form> know whether the form is currently submitting. It returns an object with pending (boolean), data (FormData), method, and action. It's from 'react-dom', not 'react'. Imported as: `import { useFormStatus } from 'react-dom';`

Real-World Analogy

Think of the 'Submit' button at an ATM. The button doesn't manage the transaction — the machine does. But the button needs to know if a transaction is in progress so it can disable itself and show 'Processing...' useFormStatus is the button asking the machine 'are we still processing?'

Syntax

```
import { useFormStatus } from 'react-dom';

const { pending, data, method, action } = useFormStatus();

// pending → true while form is submitting
// data    → FormData being submitted
// method  → 'get' or 'post'
// action  → the action function
```

Example — Submit Button Component

```
// SubmitButton.tsx — a reusable button
import { useFormStatus } from 'react-dom';

function SubmitButton({ children }) {
  const { pending } = useFormStatus();

  return (
    <button type='submit' disabled={pending}>
      {pending ? 'Submitting...' : children}
    </button>
  );
}

// Usage — SubmitButton MUST be a child of <form>
function ContactForm({ action }) {
  return (
    <form action={action}>
      <input name="message" />
      <SubmitButton>Send</SubmitButton>
    </form>
  );
}
```

■■ **Critical Gotcha:** useFormStatus only works in a CHILD component of a <form>. If you call it in the same component that renders the <form>, it returns { pending: false } always. The button must be extracted into its own component. This is the #1 gotcha with useFormStatus.

✗ Wrong

```
// WRONG: same component
function Form({ action }) {
  const { pending }
    = useFormStatus();
  // pending is ALWAYS false!
  return (
    <form action={action}>
      <button>{pending
        ? '...' : 'Send'}
      </button>
    </form>
  );
}
```

✓ Correct

```
// CORRECT: child component
function SubmitBtn() {
  const { pending }
    = useFormStatus();
  return (
    <button disabled={pending}>
      {pending ? '...' : 'Send'}
    </button>
  );
}
function Form({ action }) {
  return (
    <form action={action}>
      <SubmitBtn />
    </form>
  );
}
```

Interview Q&A

Q: Why is useFormStatus in 'react-dom' and not 'react'?

A: Because it's tied to the DOM form element — a platform-specific concept. React Native doesn't have <form>, so it doesn't need useFormStatus. React keeps platform-specific hooks in 'react-dom'.

Q: Can useFormStatus replace isPending from useActionState?

A: They serve different purposes. useActionState gives you isPending in the component that OWNS the form. useFormStatus gives you pending in a CHILD of the form. Both can exist in the same form hierarchy for different components.

#7

useOptimistic

Show instant UI updates while the server catches up — optimistic rendering built into React.

What is useOptimistic?

In plain English: useOptimistic lets you show a 'fake' updated state immediately while the real server action is still running. Once the action completes, React replaces the optimistic state with the actual server response. If the action fails, the optimistic state rolls back automatically.

Real-World Analogy

Think of pressing the 'Like' button on Instagram. The heart turns red IMMEDIATELY (optimistic update) even though the API call hasn't finished. If the API fails, the heart silently turns back to gray (rollback). This makes the app feel instant.

Syntax

```
const [optimisticState, addOptimistic] = useOptimistic(
  actualState,      // the real state (source of truth)
  updateFn          // (current, optimistic) => merged state
);
```

Example — Todo List with Optimistic Add

```
function TodoList({ todos, addTodoAction }) {
  const [optimisticTodos, addOptimistic] = useOptimistic(
    todos,
    (current, newTodo) => [
      ...current,
      { ...newTodo, sending: true }
    ]
  );

  return (
    <form action={async (formData) => {
      const title = formData.get("title");
      addOptimistic({ title, id: Date.now() });
      await addTodoAction(title); // server call
    }}>
      <input name="title" />
      <button>Add</button>
      {optimisticTodos.map(todo => (
        <p key={todo.id}
           style={{
             opacity: todo.sending ? 0.5 : 1
           }}>
          {todo.title}
          {todo.sending && ' (saving...)'}
        </p>
      ))}
    </form>
  );
}
```

Example — Like Button

```
function LikeButton({ liked, likeAction }) {
  const [optimisticLiked, setOptimisticLiked] = useOptimistic(
    liked,
    (_, newLiked) => newLiked
  );

  return (
    <form action={async () => {
      setOptimisticLiked(!optimisticLiked);
      await likeAction(); // server call
    }}>
      <button>
        {optimisticLiked ? 'Unlike' : 'Like'}
      </button>
    </form>
  );
}
```

■■ **Key Benefit:** useOptimistic works best with form actions and server actions. The optimistic state automatically reverts to the real state when the action's Promise resolves. No manual error handling or rollback code needed — React handles it.

Interview Q&A

Q: What happens if the server action fails?

A: React automatically reverts the optimistic state to the actual state (the first argument). Since the Promise rejects or the action returns an error, React re-renders with the real state. You don't need to write any rollback logic yourself.

Q: How is useOptimistic different from manually setting state before a fetch?

A: Manual optimistic updates require you to: (1) set state optimistically, (2) make the API call, (3) handle success by updating with real data, (4) handle failure by rolling back. useOptimistic does steps 1, 3, and 4 automatically. It's also concurrent-safe — it works correctly with React's concurrent renderer, while manual approaches may not.

#8

use

Read a Promise or Context during render — the only 'hook' that can be called conditionally.

What is use?

In plain English: `use` is a special React API that can unwrap Promises and read Context values during render. Unlike all other hooks, `use` can be called inside if/else blocks and loops. When given a Promise, it suspends the component until the Promise resolves (works with Suspense boundaries).

Real-World Analogy

Think of `use` as an 'await' for React components. Just like async/await lets you pause a function until a Promise resolves, `use` lets you pause a component until its data is ready. React shows a Suspense fallback while waiting.

Syntax

```
import { use } from 'react';

// Reading a Promise (suspends until resolved)
const data = use(promise);

// Reading a Context (like useContext, but conditional)
const value = use(SomeContext);
```

Example — Reading a Promise (Data Fetching)

```
// Parent creates the Promise ONCE
function UserPage({ userId }) {
  const userPromise = fetchUser(userId); // don't await

  return (
    <Suspense fallback={<Loading />}>
      <UserProfile userPromise={userPromise} />
    </Suspense>
  );
}

// Child reads the Promise with use()
function UserProfile({ userPromise }) {
  const user = use(userPromise); // suspends here

  return <h1>{user.name}</h1>;
}
```

Example — Conditional Context (Impossible with useContext)

```
function Dashboard({ isLoggedIn }) {
  // This is VALID with use() !
  if (isLoggedIn) {
    const user = use(UserContext);  // conditional!
    return <p>Welcome, {user.name}</p>;
  }

  return <p>Please log in</p>;
}

// With useContext, this would BREAK the Rules of Hooks
// because hooks can't be called conditionally.
```

■■ **Important:** `use` is technically not a 'hook' in the traditional sense — it doesn't follow the Rules of Hooks. It can be called inside conditions, loops, and after early returns. This is intentional: React designed it as a new primitive that's more flexible than hooks.

use() vs useContext() vs useEffect() for data

Aspect	use(promise)	useContext(ctx)	useEffect + useState
Can be conditional	Yes	No	No

Aspect	use(promise)	useContext(ctx)	useEffect + useState
Suspense support	Yes (suspends)	No	No (manual loading)
SSR compatible	Yes	Yes	Client-only
When data is read	During render	During render	After render (effect)
Error handling	Error Boundary	N/A	try/catch
Best for	Server Components, streaming	Global state	Legacy client-side fetching

Interview Q&A

Q: Why can use() break the Rules of Hooks?

A: Because use() is designed as a new API category, not a traditional hook. The Rules of Hooks exist to ensure React can track hook state by call order. use() doesn't need this — for Promises, it relies on the Promise identity; for Context, it reads the nearest Provider. No call-order tracking needed.

Q: Can you use() a Promise created inside the component?

A: Don't. Creating a Promise during render means a NEW Promise on every render — use() would suspend infinitely. The Promise must be created OUTSIDE the render (in a parent, a Server Component, a route loader, or cached via React.cache/useMemo). This is the #1 mistake with use().

Q: How does use() work with Next.js Server Components?

A: In Server Components, you can just use await directly (they're async functions). use() shines in Client Components that receive a Promise as a prop from a Server Component. The Server Component starts the fetch, passes the Promise down, and the Client Component reads it with use() + Suspense.

Quick Reference Cheat Sheet

Hook	Purpose	Triggers Re-render?	Key Gotcha
useTransition	Mark updates as non-urgent	Yes (isPending)	Doesn't make code faster — just prioritizes urgent updates
useDeferredValue	Defer a value to lag behind	Yes (2 renders)	Value can be stale — compare with original to show loading
useId	SSR-safe unique IDs	No	Don't use for list keys; colon in ID needs CSS escaping
useSyncExternal Store	Subscribe to external state	Yes (on change)	getSnapshot must return same ref if value hasn't changed
useActionState	Form + server action state	Yes	Renamed from useFormState; action gets prevState as 1st arg
useFormStatus	Read parent form's pending	Yes (on pending)	Must be in a CHILD of , not the same component
useOptimistic	Instant optimistic UI	Yes	Auto-reverts on failure; only works with form actions
use	Read Promise / Context	Yes (on resolve)	Not a hook — can be conditional; Promise must be stable

Priority for Interviews

Priority	Hooks	Why
Must Know	useTransition, useDeferredValue	Asked in nearly every React 18+ interview
Must Know	useActionState, useFormStatus	Core to Next.js App Router — your main stack
Good to Know	useOptimistic, use, useId	Shows awareness of React 19 patterns
Low Priority	useSyncExternalStore	Library author hook — rarely asked directly

Complete Hook Count (All 19 Hooks)

React Version	Hooks Added	Count
React 16.8 (original)	useState, useEffect, useContext, useReducer, useRef, useCallback, useMemo, useImperativeHandle, useLayoutEffect, useDebugValue	10
React 18	useTransition, useDeferredValue, useId, useSyncExternalStore, useInsertionEffect	5
React 19	useActionState, useFormStatus, useOptimistic, use	4
Total		19

Rules of Hooks (Still Apply to All Hooks Except 'use')

1. Only call hooks at the top level — never inside if/else, loops, or nested functions.
2. Only call hooks from React functions — function components or custom hooks (use_____).
3. Custom hooks must start with 'use' — useAuth, useFetch, useDebounce, etc.

Exception: The `use` API can be called conditionally and in loops. It's the only React API exempt from these rules.